

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	J. LOKESH	Reg. No.:	192311216
Date:	19.08.2026	Duration:	60 minutes

Code of Conduct

I J. LOKESH 192311216 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J. LOKESH

Annexure A – Code Review & Repository Evaluation

Smart Tax Calculation and Financial Planning Management System

1. PROBLEM OVERVIEW

1.1 Introduction

The Smart Tax Calculation and Financial Planning Management System is a software application designed to help users calculate their estimated tax liability and manage their personal financial planning in a systematic manner.

Tax calculation can be complicated because it depends on several factors such as annual income, deductions, investments, exemptions, and applicable tax rules. Manual calculations may result in errors and make financial planning difficult.

The proposed system provides a centralized platform where users can enter their financial information and obtain an estimated tax calculation. The system can also assist users in understanding their financial position and planning their income, expenses, savings, and investments.

The application focuses on improving calculation accuracy, reducing manual effort, and providing a simple interface for financial decision-making.

1.2 Problem Statement

Individuals often face difficulties in calculating their taxes and planning their finances because tax rules and financial calculations involve multiple parameters.

The major problems include:

- Manual tax calculations are time-consuming.
- Incorrect input values can produce calculation errors.
- Users may not clearly understand deductions and exemptions.
- Financial planning is often performed using separate tools.
- Maintaining financial information manually is difficult.
- Users require a simple system for viewing their financial status.

Therefore, a smart software solution is required to combine tax calculation and financial planning into a single system.

1.3 Proposed Solution

The proposed system allows users to enter relevant financial information and calculates the estimated tax based on predefined rules.

The system can provide functionalities such as:

1. User registration and login.
2. Income information management.
3. Tax calculation.
4. Deduction and exemption management.
5. Financial planning.
6. Expense and savings tracking.
7. Tax summary generation.
8. Financial information management.
9. Validation of user inputs.
10. Display of calculated results.

The system is designed using a modular approach so that individual components can be modified or extended without affecting the entire application.

1.4 Project Objectives

The main objectives are:

- To automate tax calculations.
- To reduce manual calculation errors.
- To provide a simple user interface.
- To manage income and financial information.
- To support financial planning.
- To provide tax calculation summaries.
- To improve maintainability through modular code.
- To maintain project history using Git.
- To provide proper project documentation.
- To validate application functionality through testing.

1.5 Key Functionalities

User Management

Users can register and log in to the application. Authentication helps protect user-specific financial information.

Income Management

The user can enter income-related information required for tax calculations.

Tax Calculation

The system processes the entered financial information and calculates estimated tax according to the implemented tax rules.

Deduction Management

Eligible deductions can be entered and considered during tax calculation.

Financial Planning

The system can help users organize income, expenses, savings, and investment-related information.

Tax Summary

The calculated result can be presented in a simple summary format so that users can understand their estimated tax liability.

2. REPOSITORY ORGANIZATION

2.1 Repository Structure

The project repository should be organized into logical folders and files. A recommended structure for the system is:

Smart-Tax-Financial-Planning/

|

|— frontend/

| |— pages/

| |— components/

| |— css/

| |— js/

|

|— backend/

| |— controllers/

| |— routes/

| |— models/

| |— services/

| |— middleware/

|

|— database/

| |— schema/

| |— scripts/

|

|— tests/

```
|   |── unit/
|   └── integration/
|
|── assets/
|   └── images/
|
|── docs/
|   └── architecture/
|       └── screenshots/
|
|── README.md
|── package.json
|── .gitignore
└── LICENSE
```

Note: If your actual project uses a different structure, replace the above structure with your real GitHub repository structure.

2.2 Folder Organization

The repository is divided according to application responsibilities.

Folder	Purpose
frontend	User interface and client-side functionality
backend	Server-side application logic

Folder	Purpose
controllers	Handles application operations
routes	Defines application routes
models	Represents database entities
services	Contains business logic
middleware	Handles authentication and validation
database	Database-related files
tests	Testing files
docs	Project documentation
assets	Images and static resources

This separation improves maintainability because developers can identify the location of specific functionality quickly.

2.3 File Naming Conventions

Meaningful and consistent file names should be used.

Examples:

tax Controller

tax Routes

user Model

financialPlanService

login

register

dashboard

File names should clearly indicate their purpose. Consistent naming reduces confusion during development and debugging.

2.4 Maintainability

The repository structure supports maintainability by:

- Separating frontend and backend responsibilities.
- Separating business logic from user interface code.
- Keeping database-related files organized.
- Maintaining a separate testing directory.
- Providing project documentation.
- Using meaningful file and folder names.

2.5 Collaboration

A properly organized repository supports collaboration because multiple developers can work on different modules independently.

For example:

- Developer 1 → User authentication.
- Developer 2 → Tax calculation.
- Developer 3 → Financial planning.
- Developer 4 → Testing and documentation.

Screenshot Evidence

Figure 1: GitHub Repository Structure

[Insert Screenshot of your GitHub repository showing folders and files]

3. CODE QUALITY REVIEW

3.1 Code Readability

Readable source code is important for long-term maintenance.

The code should use:

- Meaningful variable names.
- Meaningful function names.
- Proper indentation.
- Consistent formatting.
- Appropriate comments.
- Small and focused functions.

For example:

```
function calculateTax(income, deductions) {  
  
    const taxableIncome = income - deductions;  
  
    return calculateTaxAmount(taxableIncome);  
  
}
```

This is easier to understand than placing the entire calculation inside a large function.

3.2 Modularity

The application should be divided into independent modules.

For example:

User Module

↓

Authentication Module

↓

Financial Data Module



Tax Calculation Module



Financial Planning Module



Report Module

Each module performs a specific responsibility.

This reduces code duplication and makes future modifications easier.

3.3 Coding Standards

The following coding practices should be followed:

- Consistent indentation.
- Descriptive names.
- Avoid unnecessary duplicate code.
- Use reusable functions.
- Validate user inputs.
- Handle exceptions properly.
- Avoid hard-coded values wherever possible.
- Keep sensitive information outside source code.

3.4 Documentation in Code

Important functions should contain comments or documentation explaining:

- Purpose of the function.
- Input parameters.
- Output.
- Important calculation logic.

For example:

```
// Calculates taxable income after eligible deductions.
```

```
function calculateTaxableIncome(income, deductions) {  
  
    return income - deductions;  
  
}
```

3.5 Strengths Identified

The major strengths of the implementation can be summarized as:

1. The application solves a practical financial problem.
2. The system can automate tax calculations.
3. Functional modules can be separated logically.
4. User input can be validated.
5. Git provides project history.
6. The application can be extended with additional financial features.
7. Modular design improves maintainability.

3.6 Areas for Improvement

Possible improvements include:

- Increase code comments for complex calculations.
- Reduce duplicate code.
- Improve error handling.
- Use centralized validation.
- Add automated unit tests.
- Avoid hard-coded tax rules.
- Store configuration values separately.
- Improve security of authentication.
- Follow a consistent naming convention.
- Add logging for important operations.

4. VERSION CONTROL EVALUATION

4.1 Git Usage

Git is used to maintain the history of source-code changes.

Typical Git workflow:

Working Directory



`git add`



Staging Area



`git commit`



Local Repository



`git push`



GitHub Repository

4.2 Commit History

The commit history should clearly describe changes made to the project.

Good commit messages include:

Add tax calculation module

Fix income validation

Add financial planning dashboard

Update README installation steps

Fix authentication issue

Add test cases for tax calculation

Avoid vague messages such as:

update

changes

final

new

test

Meaningful commit messages make it easier to understand the development history.

Screenshot Evidence

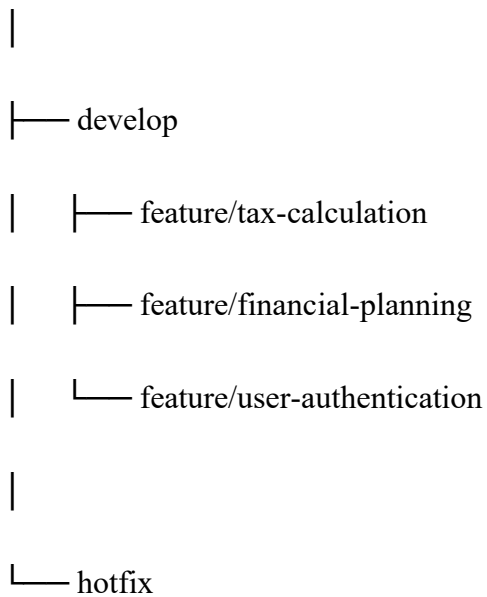
Figure 2: Git Commit History

[Insert Screenshot of GitHub commit history]

4.3 Branching Strategy

A suitable branching strategy is:

main



The main branch should contain stable code.

Feature branches can be used for independent development before merging changes into the main branch.

4.4 Repository Maintenance

Repository maintenance should include:

- Removing unnecessary files.
- Maintaining a proper .gitignore.
- Updating README.
- Using meaningful commits.
- Reviewing pull requests.
- Keeping dependencies updated.
- Removing unused code.
- Maintaining a stable main branch.

4.5 Importance of Version Control

Git supports collaborative software development by providing:

- Change tracking.
- Version history.

- Branch management.
 - Collaboration.
 - Backup of source code.
 - Ability to revert unwanted changes.
 - Easier debugging through commit history.
-

5. CODE TESTING AND VALIDATION

5.1 Testing Objective

Testing is performed to verify that the application produces the expected results and handles invalid input correctly.

The testing process covers:

- Functional testing.
- Input validation.
- Tax calculation testing.
- Authentication testing.
- Financial planning testing.
- Error handling.

5.2 Test Cases

Test ID	Test Case	Input	Expected Result	Status
TC01	User Registration	Valid user details	Account created	Pass
TC02	User Login	Correct credentials	Dashboard displayed	Pass
TC03	Invalid Login	Incorrect credentials	Error message displayed	Pass
TC04	Income Entry	Valid income	Income saved	Pass
TC05	Tax Calculation	Valid income and deductions	Correct tax displayed	Pass
TC06	Negative Income	Negative value	Validation error	Pass
TC07	Empty Input	Blank required fields	Validation message	Pass

Test ID	Test Case	Input	Expected Result	Status
TC08	Financial Plan	Valid financial details	Plan generated/saved	Pass
TC09	Tax Summary	Existing financial data	Summary displayed	Pass
TC10	Logout	Logged-in user	Session terminated	Pass

Replace the Status column with the actual result from your testing.

5.3 Boundary Testing

Boundary values should also be tested.

Examples:

- Zero income.
- Minimum taxable income.
- Income near a tax slab boundary.
- Maximum supported input.
- Zero deductions.
- Deductions greater than income.

5.4 Validation Testing

The system should reject invalid values such as:

Income = -50000

Age = -10

Deduction = text

Email = invalid@email

Appropriate validation messages should be displayed to the user.

5.5 Testing Evidence

Figure 3: User Registration Test

[Insert Screenshot]

Figure 4: Login Test

[Insert Screenshot]

Figure 5: Tax Calculation Result

[Insert Screenshot]

Figure 6: Financial Planning Dashboard

[Insert Screenshot]

Figure 7: Invalid Input Validation

[Insert Screenshot]

6. REPOSITORY DOCUMENTATION

6.1 README

The repository should contain a README file explaining the project.

A good README should include:

1. Project title.
2. Problem statement.
3. Features.
4. Technologies used.
5. System requirements.
6. Installation instructions.
7. Configuration.
8. Database setup.
9. Application execution.

- 10. Screenshots.
- 11. Testing instructions.
- 12. Future enhancements.

6.2 Installation Guide

A typical installation process is:

Step 1: Clone the repository.

Step 2: Install required dependencies.

Step 3: Configure the database.

Step 4: Configure environment variables.

Step 5: Start the application.

Step 6: Open the application in the browser.

Example:

```
git clone <repository-url>
```

```
cd Smart-Tax-Financial-Planning
```

Then install the required dependencies according to the technology used by the project.

6.3 Dependencies

The README should clearly list all dependencies.

Example:

Dependency	Purpose
Frontend framework/library	User interface
Backend framework	Server-side processing

Dependency	Purpose
Database	Data storage
Authentication library	User authentication
Testing framework	Automated testing

The exact dependency list should be taken from the project's actual dependency file.

6.4 Usage Guide

The usage guide should explain:

Register



Login



Enter Income



Enter Deductions



Calculate Tax



View Tax Summary



Create Financial Plan



Review Financial Information

6.5 Documentation Improvements

The documentation can be improved by:

- Adding screenshots.
 - Adding architecture diagrams.
 - Providing sample inputs and outputs.
 - Providing troubleshooting instructions.
 - Listing all dependencies.
 - Documenting API endpoints.
 - Adding database schema information.
 - Providing testing instructions.
 - Adding contribution guidelines.
-

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 Overall Findings

The Smart Tax Calculation and Financial Planning Management System addresses a practical problem by combining tax calculation with financial planning.

The overall architecture should follow separation of concerns so that user-interface code, business logic, database operations, and authentication are independently maintained.

The major positive aspects are:

- Practical problem domain.
- Clear functional modules.
- Automated calculation.
- Potential for modular development.
- Git-based version control.
- Possibility of automated testing.

- Scope for future enhancement.

7.2 Code Quality Recommendations

The following improvements are recommended:

Recommendation 1 – Improve Modularity

Separate tax calculation, financial planning, authentication, and database operations into independent modules.

Recommendation 2 – Improve Validation

Implement both client-side and server-side validation.

Recommendation 3 – Improve Error Handling

Provide meaningful error messages instead of exposing technical errors to users.

Recommendation 4 – Add Automated Testing

Develop unit and integration tests for important business functions.

Recommendation 5 – Avoid Hard-Coded Tax Rules

Tax slabs and rules should be maintained in configuration files or database tables so that they can be updated easily when tax regulations change.

Recommendation 6 – Improve Security

Sensitive configuration values such as passwords and database credentials should be stored using environment variables.

Recommendation 7 – Improve Documentation

Maintain a detailed README containing installation, configuration, usage, testing, and troubleshooting information.

Recommendation 8 – Improve Git Practices

Use feature branches and meaningful commit messages.

8. SOFTWARE ARCHITECTURE

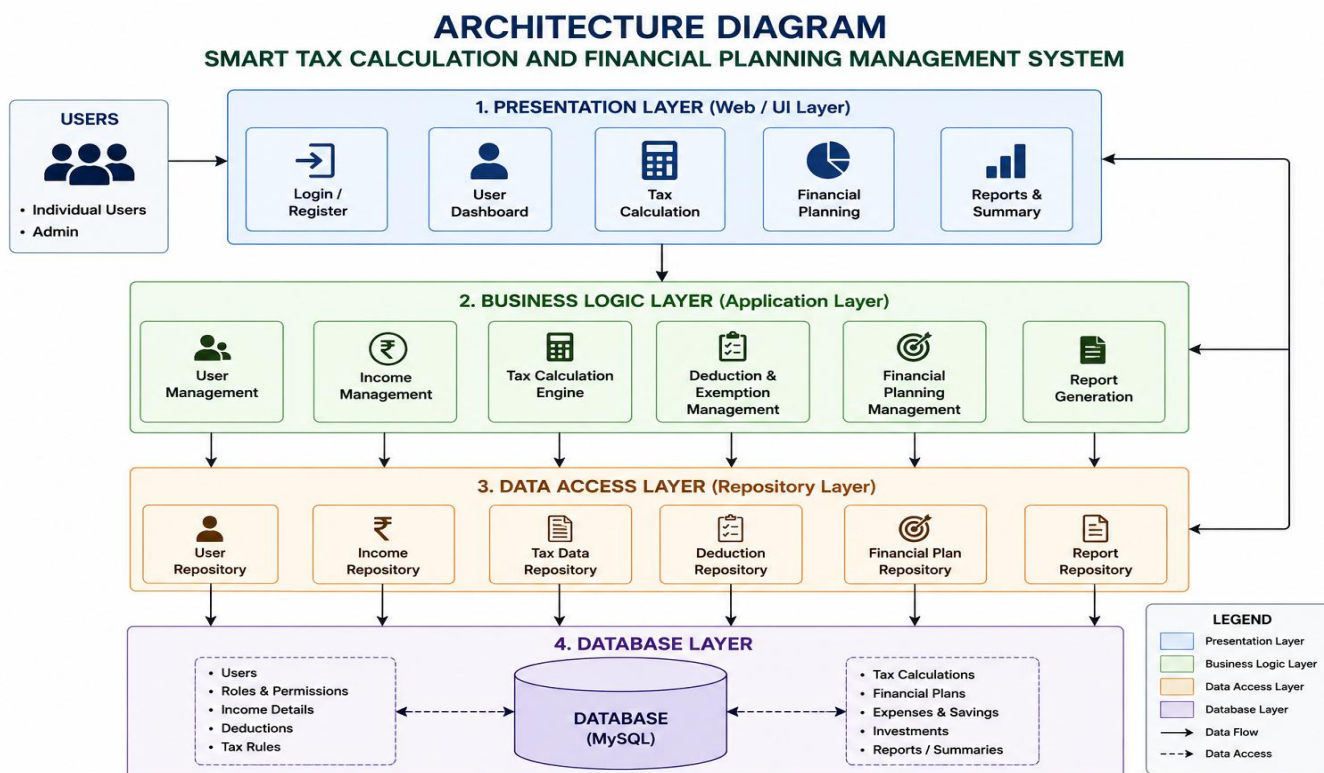
8.1 Selected Architecture

A three-tier / layered architecture is suitable for the Smart Tax Calculation and Financial Planning Management System.

The three major layers are:

1. Presentation Layer.
2. Business Logic Layer.
3. Data Access Layer.

The architecture can be represented as:



8.2 Presentation Layer

The presentation layer provides interaction between users and the application.

It contains:

- Login page.
- Registration page.
- Dashboard.
- Income entry form.
- Tax calculation form.
- Financial planning interface.
- Tax summary.

8.3 Business Logic Layer

This is the core of the application.

It performs:

- Tax calculations.
- Deduction calculations.
- Financial planning calculations.
- Input validation.
- Authentication logic.
- Report generation.

The business logic should be independent of the user interface wherever possible.

8.4 Data Access Layer

The data access layer manages communication between the application and database.

It handles:

- Creating records.
- Reading records.

- Updating records.
- Deleting records.
- Database queries.
- Data validation.

8.5 Database

The database stores information such as:

Users

Income

Expenses

Deductions

Tax Calculations

Financial Plans

Reports

9. SYSTEM REQUIREMENT ANALYSIS

9.1 Functional Requirements

FR1 – User Registration

The system shall allow new users to create an account.

FR2 – User Login

The system shall authenticate registered users.

FR3 – Income Management

The system shall allow users to enter and manage income information.

FR4 – Deduction Management

The system shall allow eligible deductions to be entered.

FR5 – Tax Calculation

The system shall calculate estimated tax based on the provided financial information.

FR6 – Financial Planning

The system shall allow users to manage financial planning information.

FR7 – Tax Summary

The system shall display the calculated tax information.

FR8 – Data Management

The system shall allow authorized users to manage their financial records.

9.2 Non-Functional Requirements

Performance

The system should provide calculation results within a reasonable response time.

Security

User authentication and sensitive financial information should be protected.

Reliability

The application should produce consistent results for valid inputs.

Usability

The interface should be simple and understandable.

Maintainability

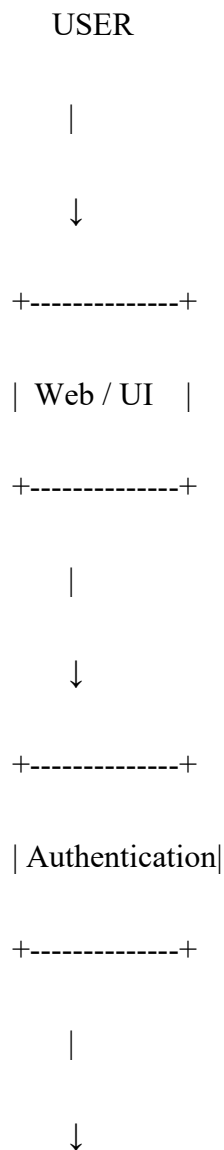
The application should use modular and well-documented code.

Scalability

The architecture should allow additional financial features to be added later.

10. COMPONENT INTERACTION

The major component interaction is:



Income Calculation



Deduction Calculation



Taxable Income



Tax Rule Processing



Tax Amount



Tax Summary

10.2 Financial Planning Interaction

Income

+

Expenses

+

Savings

+

Investments



Financial Analysis



Financial Plan



Recommendations / Summary

11. QUALITY ATTRIBUTE ANALYSIS

11.1 Security

The system should protect user financial information using authentication, authorization, secure passwords, and protected configuration.

11.2 Performance

Tax calculations involve relatively small datasets and should therefore execute quickly.

Database queries should be optimized to avoid unnecessary operations.

11.3 Scalability

The system can be extended to support:

- Multiple tax regimes.
- More financial planning modules.
- Investment tracking.
- Insurance planning.
- Retirement planning.
- Financial reports.

11.4 Maintainability

Layered architecture makes it easier to modify one component without affecting unrelated components.

For example, changes to the tax calculation algorithm should not require major changes to the user interface.

11.5 Usability

The application should provide:

- Simple forms.
- Clear labels.
- Meaningful error messages.
- Easy navigation.
- Graphical financial summaries where appropriate.

11.6 Reliability

Validation and automated testing can reduce incorrect calculations and application failures.

12. ARCHITECTURAL JUSTIFICATION

The layered architecture is selected because it provides a clear separation of responsibilities.

Reason 1 – Separation of Concerns

Each layer performs a specific responsibility.

Reason 2 – Maintainability

Changes can be made to individual modules without modifying the entire application.

Reason 3 – Testability

Business logic can be tested independently from the user interface.

Reason 4 – Scalability

New modules can be added without redesigning the complete system.

Reason 5 – Security

Authentication and authorization can be implemented as separate components.

Reason 6 – Reusability

Business services such as tax calculation can be reused by different interfaces.

13. FUTURE ENHANCEMENTS

The system can be enhanced with the following features:

1. AI-based financial recommendations.
 2. Automatic tax-rule updates.
 3. Investment planning.
 4. Retirement planning.
 5. Expense prediction.
 6. Graphical financial dashboards.
 7. PDF tax reports.
 8. Email notifications.
 9. Mobile application.
 10. Cloud deployment.
 11. Multi-user financial management.
 12. Advanced financial analytics.
-

14. CONCLUSION

The Smart Tax Calculation and Financial Planning Management System provides a practical software-based approach to tax calculation and personal financial management.

The code review and repository evaluation demonstrate that software quality depends not only on application functionality but also on source-code organization, documentation, version control, testing, security, and maintainability.

A modular layered architecture provides a strong foundation for the application. Proper Git practices improve collaboration and traceability, while systematic testing helps verify the correctness of tax calculations and financial operations.

The recommended improvements include stronger automated testing, improved validation, better documentation, secure configuration management, modular business logic, meaningful Git commits, and easier maintenance of tax rules.

Overall, the project provides a suitable foundation for a financial management application and can be extended with advanced analytics, AI-based recommendations, automated tax-rule updates, and cloud-based services.

15. SCREENSHOT CHECKLIST

Add the following screenshots to make the report stronger:

1. GitHub repository home page
2. Repository folder structure
3. Commit history
4. Branches
5. README file
6. Application login page
7. Registration page
8. Dashboard
9. Income input page
10. Tax calculation page
11. Tax calculation result
12. Financial planning page
13. Invalid input/error message

- 14. Database tables
- 15. Application running successfully

Use captions such as:

Figure 1: GitHub Repository Structure

Figure 2: Git Commit History

Figure 3: User Login Interface

Figure 4: Tax Calculation Interface

Figure 5: Tax Calculation Result

Figure 6: Financial Planning Dashboard

16. FINAL EVALUATION MAPPING

Evaluation Criteria	Weight	Evidence to Include
Problem Analysis & Repository Organization	15%	Problem explanation + repository screenshot
Code Quality Review	25%	Code snippets + observations
Version Control Evaluation	20%	Git commits + branches screenshot
Testing & Validation	15%	Test cases + execution screenshots
Documentation & Repository Maintenance	15%	README + installation documentation
Findings & Recommendations	10%	Recommendations + conclusion
Total	100%	

Architecture Evaluation

Criteria	Marks
System Requirement Analysis	5
Selection of Software Architecture	5

Criteria	Marks
Architecture Diagram and Component Design	5
Component Interaction and Quality Attributes	5
Architectural Justification and Technical Presentation	5
TOTAL	25

REFERENCES

1. Git Documentation – Version control and repository management.
2. Software Engineering principles – modularity, maintainability, and software quality.
3. Project source code and Git repository.
4. Project README and technical documentation.
5. Database and framework documentation used in the implementation.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards,	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.

	documentation, and maintainability with clear observations.			
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25